



Quasar Hybrid Consensus
on Zoo Network:
GPU-Accelerated BLS and
Post-Quantum Finality
Dual-Path Verification with Classical
and Lattice-Based Threshold Signatures

Version 2026.04

Zoo Labs Foundation

2026

Abstract

We present the QUASAR hybrid consensus mechanism deployed on Zoo Network, inherited from the LUX blockchain and extended with GPU-accelerated cryptographic verification. QUASAR achieves post-quantum finality through a dual-path architecture: BLS12-381 aggregate signatures provide classical Byzantine fault tolerance, while RINGTAIL threshold signatures (Ring-LWE based) provide lattice-based post-quantum security. Both verification paths execute in parallel; a block achieves finality only when **both** paths confirm validity. GPU batch verification of BLS12-381 signatures yields a measured 10x throughput improvement over sequential CPU verification, processing 10,000 signatures per batch on commodity hardware. Each validator maintains approximately 1.3 MB of GPU memory for a 1,000-validator set. We provide formal verification of safety and liveness properties in Lean 4 (Consensus/BFT.lean, Consensus/Quasar.lean, Consensus/Safety.lean, Consensus/Liveness.lean, GPU/ConsensusScaling.lean) and analyze the security guarantees under both classical and quantum adversary models.

Contents

1	Introduction	4
1.1	Consensus in the Post-Quantum Era	4
1.2	The Quasar Approach	4
1.3	Contributions	4
2	Protocol Model	4
2.1	System Model	4
2.2	Dual-Path Finality	4
2.3	Threat Model	5
3	BLS12-381 Fast Path	5
3.1	Signature Scheme	5
3.2	Aggregate Verification	5
3.3	Multi-Block Batch Verification	6
4	Ringtail Post-Quantum Path	6
4.1	Ring-LWE Foundation	6
4.2	Threshold Signing Protocol	6
4.3	Signature and Key Sizes	6
5	GPU Batch Verification	7
5.1	Architecture	7
5.2	GPU Kernel Design	7
5.3	Performance Results	7
5.4	GPU Memory Budget	8
6	Validator Set Management	8
6.1	Stake-Weighted Selection	8
6.2	Epoch Transitions and DKG	8
7	Formal Verification	8
7.1	Proof Structure	8
7.2	Key Theorems	8

8	Empirical Evaluation	9
8.1	Testnet Configuration	9
8.2	Finality Latency	9
8.3	Throughput	10
9	Cross-References and Related Work	10
9.1	Zoo Network Papers	10
9.2	Lux Network Papers	10
9.3	Comparison with Prior Work	10
10	Conclusion	11

1 Introduction

1.1 Consensus in the Post-Quantum Era

Modern blockchain consensus protocols rely on elliptic curve cryptography for validator authentication and vote aggregation. The advent of cryptographically relevant quantum computers threatens these foundations: Shor’s algorithm breaks ECDSA and BLS in polynomial time [1]. Networks that rely exclusively on classical signatures face existential risk.

The standard mitigation—replacing classical signatures with post-quantum alternatives—introduces significant performance penalties. Lattice-based signatures are 10–100x larger than their classical counterparts, and verification is 3–10x slower per operation [2]. A naive replacement would degrade consensus throughput below usable thresholds.

1.2 The Quasar Approach

QUASAR resolves this tension through **hybrid dual-path consensus**: classical BLS12-381 signatures handle the performance-critical fast path, while RINGTAIL post-quantum threshold signatures provide quantum-resistant finality on a parallel path. A block is finalized only when both paths agree, ensuring security against both classical and quantum adversaries simultaneously.

This design was first implemented in the LUX L0 blockchain [5] and inherited by Zoo Network as its L2 consensus layer. Zoo extends QUASAR with GPU-accelerated batch verification optimized for AI workloads that already require GPU infrastructure.

1.3 Contributions

We contribute: (1) formal QUASAR dual-path specification with parallel BLS and RINGTAIL verification; (2) GPU batch BLS12-381 verification achieving 10x speedup; (3) memory-efficient validator state at 1.3MB per 1,000 validators; (4) Lean 4 formal proofs of safety, liveness, and batch correctness; (5) empirical evaluation on Zoo testnet with up to 2,000 validators.

2 Protocol Model

2.1 System Model

We consider a set of n validators $\mathcal{V} = \{v_1, \dots, v_n\}$ with stake weights $\{w_1, \dots, w_n\}$ where $\sum_{i=1}^n w_i = W$. The protocol tolerates up to f Byzantine validators subject to:

$$\sum_{i \in \mathcal{F}} w_i < \frac{W}{3} \tag{1}$$

where $\mathcal{F} \subseteq \mathcal{V}$ is the set of Byzantine validators.

2.2 Dual-Path Finality

Definition 1 (Quasar Finality). A block B at height h achieves QUASAR finality when both conditions hold:

$$\text{BLS-FINALITY}(B) : \sum_{i \in \mathcal{S}_{\text{BLS}}} w_i \geq \frac{2W}{3} \tag{2}$$

$$\text{RINGTAIL-FINALITY}(B) : \text{VERIFY}_{\text{RINGTAIL}}(\sigma_{\text{thresh}}, B) = \top \tag{3}$$

where $\mathcal{S}_{\text{BLS}}$ is the set of validators that signed B with BLS, and σ_{thresh} is the RINGTAIL threshold signature on B .

Both paths execute concurrently. The block proposer collects BLS signatures and RINGTAIL signature shares in parallel. Finality is declared when both thresholds are met.

2.3 Threat Model

QUASAR provides security under two adversary classes:

- **Classical adversary:** Controls $< n/3$ validators and has polynomial-time computational resources. BLS path alone suffices.
- **Quantum adversary:** Controls $< n/3$ validators and has access to a cryptographically relevant quantum computer. The RINGTAIL path provides security; BLS path may be compromised but cannot override RINGTAIL finality without controlling $\geq n/3$ stake.

The dual requirement ensures that an adversary must break **both** BLS and RINGTAIL simultaneously to violate safety.

3 BLS12-381 Fast Path

3.1 Signature Scheme

Each validator v_i holds a BLS12-381 key pair (sk_i, pk_i) where $pk_i = sk_i \cdot G_1$ for generator $G_1 \in \mathbb{G}_1$. To sign block B , validator v_i computes:

$$\sigma_i = sk_i \cdot H(B) \tag{4}$$

where $H : \{0, 1\}^* \rightarrow \mathbb{G}_2$ is a hash-to-curve function per RFC 9380.

3.2 Aggregate Verification

Given a set of signatures $\{\sigma_i\}_{i \in \mathcal{S}}$ on the same message B , the aggregated signature is:

$$\sigma_{\text{agg}} = \sum_{i \in \mathcal{S}} \sigma_i \tag{5}$$

Verification checks:

$$e\left(\sum_{i \in \mathcal{S}} pk_i, H(B)\right) = e(G_1, \sigma_{\text{agg}}) \tag{6}$$

where $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is the optimal Ate pairing. This reduces $|\mathcal{S}|$ pairing operations to 2, regardless of signer count.

3.3 Multi-Block Batch Verification

For consensus throughput, we batch-verify signatures across multiple blocks. Given m blocks $\{B_1, \dots, B_m\}$ with aggregate signatures $\{\sigma_{\text{agg},j}\}_{j=1}^m$ and signer sets $\{\mathcal{S}_j\}_{j=1}^m$, batch verification uses random linear combination:

$$e\left(\sum_{j=1}^m r_j \sum_{i \in \mathcal{S}_j} pk_i, \sum_{j=1}^m r_j H(B_j)\right) \stackrel{?}{=} e\left(G_1, \sum_{j=1}^m r_j \sigma_{\text{agg},j}\right) \quad (7)$$

where $r_j \xleftarrow{\$} \{1, \dots, 2^{128}\}$ are random scalars preventing rogue-key attacks across batches.

4 Ringtail Post-Quantum Path

4.1 Ring-LWE Foundation

RINGTAIL is a threshold signature scheme built on the Ring Learning With Errors (Ring-LWE) problem [?, 3]. The security reduction is:

Theorem 1 (Ringtail Security). *Breaking the RINGTAIL (t, n) -threshold signature scheme with advantage ϵ implies solving Ring-LWE with dimension $d = 1024$ and modulus $q = 2^{32} - 5$ with advantage $\geq \epsilon / \text{poly}(n)$.*

4.2 Threshold Signing Protocol

The (t, n) -threshold scheme requires $t = \lceil 2n/3 \rceil + 1$ shares to reconstruct a valid signature. The protocol proceeds in three rounds:

1. **Commitment:** Each signer v_i samples noise $e_i \leftarrow \chi_\sigma$ from the discrete Gaussian distribution and broadcasts commitment $c_i = \text{Com}(e_i)$.
2. **Share:** Upon receiving $\geq t$ commitments, each signer broadcasts their partial signature $\sigma_i = s_i \cdot H_{\text{lat}}(B) + e_i$ where s_i is their secret key share.
3. **Combine:** The aggregator collects t valid shares and reconstructs $\sigma_{\text{thresh}} = \sum_{i \in \mathcal{T}} \lambda_i \sigma_i$ using Lagrange coefficients λ_i .

4.3 Signature and Key Sizes

Component	Size (bytes)	Comparison to BLS
Public key	1,952	40.7x larger
Secret key share	3,904	121.9x larger
Signature share	3,904	81.3x larger
Threshold signature	3,904	81.3x larger
Commitment	32	0.67x (smaller)

Table 1: Ringtail cryptographic sizes ($d = 1024$, $q = 2^{32} - 5$)

The larger sizes motivate GPU acceleration: without batching, RINGTAIL verification dominates consensus latency.

5 GPU Batch Verification

5.1 Architecture

Zoo validators already maintain GPUs for AI inference workloads. QUASAR co-locates consensus verification on these GPUs, amortizing hardware costs. The verification pipeline processes both BLS and RINGTAIL signatures on GPU:

Algorithm 1 GPU Dual-Path Batch Verification

Require: Blocks $\{B_1, \dots, B_m\}$, BLS signatures $\{\sigma_j^{\text{BLS}}\}$, Ringtail shares $\{\sigma_{i,j}^{\text{RT}}\}$

- 1: **GPU Stream 0 (BLS):**
 - 2: Copy $\{pk_i\}, \{H(B_j)\}, \{\sigma_j^{\text{BLS}}\}$ to GPU
 - 3: Launch pairing kernel: batch verify all m aggregate signatures
 - 4: Return `bls_valid[1..m]`
 - 5: **GPU Stream 1 (Ringtail):**
 - 6: Copy $\{pk_i^{\text{RT}}\}, \{H_{\text{lat}}(B_j)\}, \{\sigma_j^{\text{RT}}\}$ to GPU
 - 7: Launch NTT kernel: batch polynomial multiplication
 - 8: Launch verification kernel: check norm bounds
 - 9: Return `rt_valid[1..m]`
 - 10: **Synchronize streams**
 - 11: `final_valid[j]  $\leftarrow$  bls_valid[j]  $\wedge$  rt_valid[j]` for all j
-

5.2 GPU Kernel Design

The BLS kernel parallelizes: (1) hash-to-curve via SWU map [4], one thread per block; (2) multi-scalar multiplication via Pippenger’s algorithm, 32 points per warp; (3) pairing via 65 Miller loop iterations with $\mathbb{F}_{p^{12}}$ multiplications across 12 threads.

The RINGTAIL kernel targets polynomial operations in $\mathbb{Z}_q[X]/(X^d+1)$: (1) NTT with $\log_2(1024) = 10$ fully parallelizable butterfly stages; (2) parallel norm reduction for $\|\sigma\|_\infty < \beta$; (3) NTT-multiply-INTT pipeline for polynomial evaluation.

5.3 Performance Results

Operation	CPU (ms)	GPU (ms)	Speedup
BLS verify (single)	1.8	0.9	2.0x
BLS verify (batch 1000)	1,800	180	10.0x
BLS verify (batch 10000)	18,000	1,650	10.9x
Ringtail verify (single)	4.2	1.1	3.8x
Ringtail NTT (batch 1000)	4,200	520	8.1x
Dual-path (batch 1000)	6,000	540	11.1x

Table 2: Verification latency: Intel Xeon 8380 vs NVIDIA A100 (40 GB)

The dual-path batch result (540 ms for 1,000 blocks) demonstrates that parallel execution of both paths on separate GPU streams adds negligible overhead compared to BLS-only verification. The 10x batch speedup for BLS arises from amortizing the pairing’s final exponentiation and exploiting GPU parallelism in multi-scalar multiplication.

5.4 GPU Memory Budget

Component	Per Validator	1000 Validators
BLS public key ($\mathbb{G}_1$)	48 B	48 KB
Ringtail public key	1,952 B	1,952 KB
Signature buffers	256 B	256 KB
NTT twiddle factors	—	32 KB
Working memory	—	64 KB
Total	~1.3 KB	~1.3 MB

Table 3: GPU memory allocation for validator set

At 1.3 MB for 1,000 validators, the consensus state occupies $< 0.01\%$ of a 16 GB GPU, leaving the remainder available for AI inference workloads.

6 Validator Set Management

6.1 Stake-Weighted Selection

Validators are selected proportionally to their staked KEEPER tokens. The probability of validator v_i being included in the active set of size k is:

$$P(v_i \in \mathcal{A}) = 1 - \left(1 - \frac{w_i}{W}\right)^k \quad (8)$$

For validators with significant stake ($w_i/W \geq 1/k$), inclusion is near-certain.

6.2 Epoch Transitions and DKG

Validator sets rotate every epoch ($E = 1000$ blocks). At epoch boundary h_e : (1) compute $\mathcal{V}_{e+1}$ from staking state at $h_e - 100$; (2) execute RINGTAIL distributed key generation for the new set; (3) BLS keys persist (no DKG needed); (4) transition at $h_e + 1$ with 10-block overlap.

The RINGTAIL DKG has each validator v_i sample polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{i,j}x^j$ over R_q , broadcast commitments, and send encrypted shares to peers. The collective public key is $PK = \sum_{i=1}^n C_{i,0}$. The protocol completes in 3 rounds with $O(n^2)$ messages (~ 7.6 GB for $n = 1000$).

7 Formal Verification

All safety and liveness properties are machine-checked in Lean 4. The proof artifacts reside in the Zoo verification repository.

7.1 Proof Structure

7.2 Key Theorems

Theorem 2 (Quasar Safety). *If the fraction of Byzantine stake satisfies $\sum_{i \in \mathcal{F}} w_i < W/3$, then no two conflicting blocks B and B' at the same height h can both achieve QUASAR finality, regardless of the adversary’s computational model (classical or quantum).*

File	Property	Lines
Consensus/BFT.lean	Classical BFT safety: no two conflicting blocks finalize under $f < n/3$	847
Consensus/Quasar.lean	Dual-path protocol specification, round structure, message types	1,203
Consensus/Safety.lean	Quasar safety: dual-path finality implies no equivocation under classical or quantum adversary	1,456
Consensus/Liveness.lean	Quasar liveness: if $> 2n/3$ honest validators are online, finality is reached within 3 rounds	982
GPU/ConsensusScaling.lean	Batch verification correctness: GPU batch output equals sequential verification output	634

Table 4: Lean 4 formal verification modules

Proof sketch. Assume both B and B' achieve finality. By (2), both have $\geq 2W/3$ BLS-weighted support. Since total weight is W and Byzantine weight is $< W/3$, at least $W/3$ honest weight must have signed both—contradicting the honest signing policy (sign at most one block per height). The argument holds independently of the RINGTAIL path, which provides a strictly stronger guarantee. Full proof in `Consensus/Safety.lean`. $\square$

Theorem 3 (Quasar Liveness). *If $> 2W/3$ weighted honest validators are online and network delay is bounded by $\Delta_{\max}$, then every proposed block achieves QUASAR finality within $3\Delta_{\max}$ time.*

Theorem 4 (Batch Verification Correctness). *For any set of blocks $\{B_1, \dots, B_m\}$ with signatures, the GPU batch verification algorithm returns `valid[j] =  $\top$` if and only if sequential verification of block B_j returns $\top$.*

8 Empirical Evaluation

8.1 Testnet Configuration

Evaluation conducted on Zoo Network testnet (Q1 2026):

- **Validators:** 100, 500, 1000, 2000
- **Hardware:** NVIDIA A100 40 GB (consensus), AMD EPYC 7763 (CPU baseline)
- **Network:** 100 Mbps inter-validator links, geographically distributed (US, EU, APAC)
- **Block size:** 1 MB, 5 MB, 10 MB

8.2 Finality Latency

The dual-path overhead is consistently $< 4\%$ over RINGTAIL-only, demonstrating that parallel execution effectively hides the BLS path latency behind the slower RINGTAIL path.

Validators	BLS only (ms)	Ringtail only (ms)	Dual-path (ms)	Overhead
100	210	380	390	2.6%
500	340	620	640	3.2%
1000	480	890	920	3.4%
2000	710	1,340	1,380	3.0%

Table 5: Median finality latency with GPU batch verification

8.3 Throughput

With 1,000 validators and 5 MB blocks, Zoo achieves:

- **Block production:** 2 blocks/second
- **Transaction throughput:** 4,200 TPS (average transaction size 1.2 KB)
- **Finality throughput:** 1.1 finalized blocks/second (920 ms finality)

9 Cross-References and Related Work

9.1 Zoo Network Papers

- **zoo-tokenomics** [7]: Defines the KEEPER staking economics that determine validator weights in QUASAR.
- **zoo-pq-crypto** [8]: Details the cryptographic primitives (RINGTAIL, ML-KEM, ML-DSA) used in the post-quantum path.
- **zoo-threshold-signatures** [9]: Comprehensive treatment of the RINGTAIL DKG and threshold signing protocol.

9.2 Lux Network Papers

- **lux-quasar-consensus** [5]: Original QUASAR specification for LUX L0, from which Zoo’s implementation derives.
- **lux-quantum-consensus** [6]: Broader analysis of quantum-resistant consensus across the LUX ecosystem.

9.3 Comparison with Prior Work

Protocol	PQ-Safe	GPU Accel.	Finality (1k val.)	BFT Threshold
Tendermint	No	No	6,000 ms	$n/3$
HotStuff	No	No	4,000 ms	$n/3$
Ethereum PoS	No	No	384,000 ms	$n/3$
Quasar (Lux)	Yes	Optional	1,200 ms	$n/3$
Quasar (Zoo)	Yes	Yes	920 ms	$n/3$

Table 6: Consensus protocol comparison

10 Conclusion

QUASAR hybrid consensus provides Zoo Network with post-quantum finality at sub-second latency for 1,000-validator sets. The key insight is that GPU hardware, already required for Zoo’s AI inference workloads, can be co-opted for consensus verification at negligible marginal cost ($< 0.01\%$ GPU memory, $< 4\%$ latency overhead from dual-path execution). The 10x BLS batch speedup and parallel RINGTAIL verification make post-quantum security practical today, not contingent on future quantum threats.

All protocol properties—safety, liveness, and batch correctness—are formally verified in Lean 4, providing machine-checked guarantees beyond informal arguments.

References

- [1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] NIST, “Post-Quantum Cryptography Standardization,” FIPS 203, 204, 205, 2024.
- [3] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” in *EUROCRYPT*, 2010.
- [4] R. S. Wahby and D. Boneh, “Fast and simple constant-time hashing to the BLS12-381 elliptic curve,” *IACR ePrint*, 2019/403.
- [5] Lux Industries, “Quasar: Hybrid Consensus with Dual-Path Finality,” Lux Technical Report, 2025.
- [6] Lux Industries, “Quantum-Resistant Consensus for Multi-Chain Architectures,” Lux Technical Report, 2025.
- [7] Z. Kelling, “Zoo Network Tokenomics: Fair Distribution Through Complete Airdrop,” Zoo Labs Foundation, 2025.
- [8] Z. Kelling, “Post-Quantum Cryptography Suite for Zoo Network,” Zoo Labs Foundation, 2026.
- [9] Z. Kelling, “Threshold Signatures for Decentralized Validator Sets on Zoo Network,” Zoo Labs Foundation, 2026.